

Deterministic Optimization

Nonlinear Optimization
Modeling – Approximation
and Fitting

Andy Sun

Assistant Professor

Stewart School of Industrial and Systems Engineering

Image Compression, Constrained
Least Squares, and SVD

Nonlinear Optimization Modeling

Learning Objectives for this lesson

- Formulate image compression as a low rank approximation problem
- Formulate the approximation problem a constrained least squares problem
- Solve it by SVD

Digital Images and Representation

- Consider this image of 320-by-200 black-and-white pixels
- Each pixel is represented by a gray scale number ranging from 0 (black) to 1 (white)
- Each number is stored as a double-precision floating-point value of 8 bytes (64 bits)
- The image is stored as a 320-by-200 matrix of 8-bytes numbers
- That is $320 \times 200 \times 8 = 51.2\text{kB}$ data
- It does not sound like a big amount of data
- But...

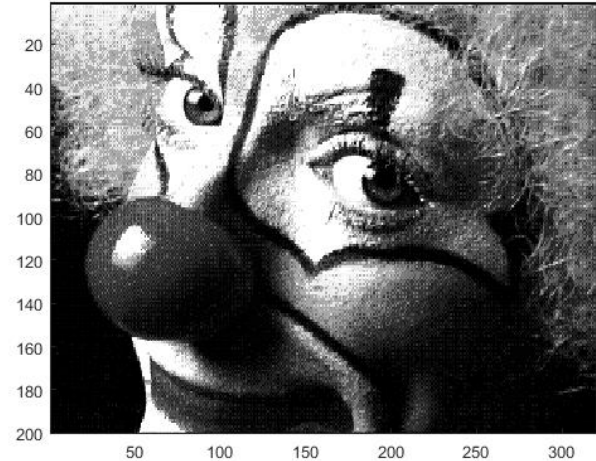


Image Compression

- Imagine you have a million much bigger colored pictures, e.g. taken as images from an experiment in a biology lab
- How are you going to store and transmit them through wifi to your collaborators?
- A solution is to do **image compression**:
 - **Instead of storing $m \cdot n$ numbers, we store much fewer numbers, from which we can still approximately reconstruct the image**
- The key word is **approximation**

Image Compression as Approximation

- The image is represented by an m -by- n matrix A of full rank
- We want to find a “simpler” matrix $\tilde{A}$ that “approximates” A well
- A good criterion for a “simple” matrix is the **rank** of the matrix
 - We require $\text{rank}(\tilde{A}) = r \ll \text{rank}(A)$
 - $\tilde{A}$ is called a **low rank matrix**
- A good criterion for “approximation” is that $\tilde{A}$ is close to A in norm:
 - That is: $\|\tilde{A} - A\|_2$ **is small**

Low Rank Matrix Approximation

- Put things together, mathematically, image compression can be formulated as the following constrained optimization problem:

$$\begin{aligned} \min_{\tilde{A}} & ||\tilde{A} - A||_2 \\ \text{s.t. } & \text{rank}(\tilde{A}) = r \\ & \tilde{A} \in R^{m \times n} \end{aligned}$$

- This is called a **low rank matrix approximation** problem
- It is nothing but a **least squares** problem with a constraint!

Low Rank Approximation by SVD

- SVD is ready to help with the following theorem:

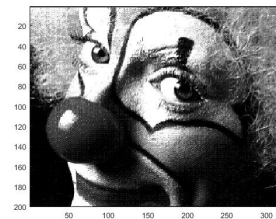
Theorem: Let $A = \sum_{i=1}^n \sigma_i u_i v_i^\top = U \Sigma V^\top$ be the SVD of the m -by- n matrix A with $m \geq n$, where $U = [u_1, \dots, u_n]$, $V = [v_1, \dots, v_n]$, and $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n)$ with $\sigma_1 \geq \dots \geq \sigma_n$. Then the rank r matrix $\tilde{A}$ closest to A in $\|\cdot\|_2$ norm is given by $\tilde{A} = \sum_{i=1}^r \sigma_i u_i v_i^\top = U \Sigma_r V^\top$, where $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0)$. Furthermore, $\|\tilde{A} - A\|_2 = \sigma_{r+1}$.

- Remark: This is an **amazing** theorem! The low rank approximation problem is highly nonconvex, but the above theorem suggests an almost closed form solution.

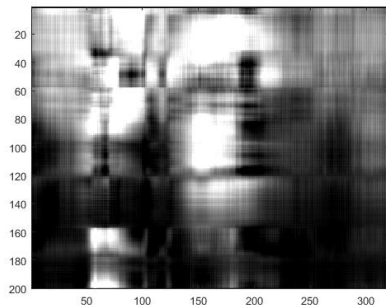
Low Rank Approximation Algorithm

- Algorithm for rank r approximation of m -by- n A :
 - Do SVD on A
 - Keep the first r largest singular values and associated left and right singular vectors
 - Construct $\tilde{A} = \sum_{i=1}^r \sigma_i u_i v_i^\top$
- Space complexity of the algorithm:
 - Without algorithm, need to store $m \cdot n$ numbers in A
 - With algorithm: store $m \cdot r$ numbers in $u_1, \dots, u_r$ and $n \cdot r$ numbers in $v_1, \dots, v_r$, thus in total $(m + n) \cdot r$ numbers
 - Compression ratio: $(m + n) \cdot r / (m \cdot n)$

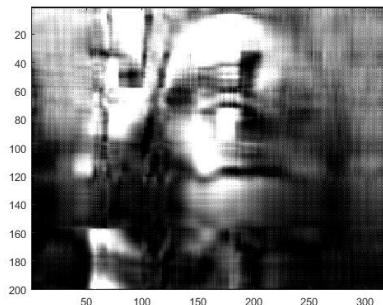
Example



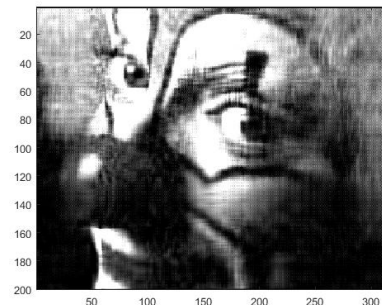
- Rank 5 approx:



- Rank 10 approx:



- Rank 20 approx:



- Relative error:

$$\frac{\sigma_6}{\sigma_1} = 13.12\%$$

- Compression ration:

$$\frac{520 * 5}{64000} = 4.06\%$$

- Relative error:

$$\frac{\sigma_{11}}{\sigma_1} = 7.66\%$$

- Compression ration:

$$\frac{520 * 10}{64000} = 8.12\%$$

- Relative error:

$$\frac{\sigma_{21}}{\sigma_1} = 4.03\%$$

- Compression ration:

$$\frac{520 * 20}{64000} = 16.24\%$$

Summary

- We learned:
 - How to formulate image compression as an approximation problem
 - How to solve this problem by SVD